

■ LEVEL 7 • BINARY SEARCH

Pattern Recognition • Problem Practice • Optimal C++ • Complexity

Pattern Trigger	Sorted search space or a monotonic answer. Each decision removes about half the possibilities.
Reference Examples	Binary Search • Search in Rotated Sorted Array • First/Last Position • Koko Eating Bananas • Capacity to Ship Packages
Problems	6 focused problems in this level
Code Standard	C++ • visible nested indentation • syntax-colored keywords/functions/comments/strings

Reference framework: the user's **DSA Approach Handbook**. The handbook defines the trigger, why, key thought, complexity, and representative practice problems; this continuation turns each level into a problem-by-problem practice book.

LEVEL 7 • EXPANDED PROBLEM INDEX

The original 6 core problems are retained. Four additional problems are added below, giving 10 total problems in this level.

#	Problem	Pattern	LeetCode
1	Binary Search	Binary Search	#704
2	Search Insert Position	Binary Search	#35
3	First and Last Position	Lower/Upper Bound	#34
4	Search in Rotated Sorted Array	Modified Binary Search	#33
5	Koko Eating Bananas	Binary Search on Answer	#875
6	Capacity To Ship Packages Within D Days	Binary Search on Answer	#1011
7	Search in Rotated Sorted Array	Binary Search	33
8	Find First and Last Position	Boundary Binary Search	34
9	Find Minimum in Rotated Sorted Array	Binary Search	153
10	Koko Eating Bananas	Binary Search on Answer	875

Extension rule: Problems 7–10 are additional practice chosen to strengthen the same pattern. They are clearly separated from the reference handbook's original representative set.

PROBLEM 1 Binary Search

Pattern: Binary Search

LeetCode: #704

■ Input	sorted integer array and target
■ Output	index of target, or -1
■ Constraints	array sorted ascending
■ Main Idea / Pattern	Compare with middle and discard half of the search space.
■■ Time Complexity	$O(\log n)$
■ Space Complexity	$O(1)$

■ Optimal C++ Code

```
int search(vector<int>& nums, int target) {  
    int l = 0, r = nums.size() - 1;  
    while (l <= r) {  
        int mid = l + (r - l) / 2;  
        if (nums[mid] == target) return mid;  
        if (nums[mid] < target) l = mid + 1;  
        else r = mid - 1;  
    }  
    return -1;  
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 2 Search Insert Position

Pattern: Binary Search

LeetCode: #35

■ Input	sorted distinct array and target
■ Output	index where target is or should be inserted
■ Constraints	sorted ascending
■ Main Idea / Pattern	Find the first position whose value is $\geq$ target.
■ Time Complexity	$O(\log n)$
■ Space Complexity	$O(1)$

■ Optimal C++ Code

```
int searchInsert(vector<int>& nums, int target) {
    int l = 0, r = nums.size();
    while (l < r) {
        int mid = l + (r - l) / 2;
        if (nums[mid] < target) l = mid + 1;
        else r = mid;
    }
    return l;
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 3 First and Last Position

Pattern: Lower/Upper Bound

LeetCode: #34

■ Input	sorted array and target
■ Output	first and last index of target
■ Constraints	sorted array
■ Main Idea / Pattern	Run binary search twice: first position $\geq$ target and first position $>$ target.
■■ Time Complexity	$O(\log n)$
■ Space Complexity	$O(1)$

■ Optimal C++ Code

```
vector<int> searchRange(vector<int>& nums, int target) {  
    auto lb = lower_bound(nums.begin(), nums.end(), target);  
    auto ub = upper_bound(nums.begin(), nums.end(), target);  
  
    if (lb == nums.end() || *lb != target)  
        return {-1, -1};  
  
    return {int(lb - nums.begin()), int(ub - nums.begin()) - 1};  
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 4 Search in Rotated Sorted Array

Pattern: Modified Binary Search

LeetCode: #33

■ Input	rotated sorted array and target
■ Output	target index or -1
■ Constraints	distinct values
■ Main Idea / Pattern	At every step, one half is sorted. Decide whether target lies in that half.
■ Time Complexity	$O(\log n)$
■ Space Complexity	$O(1)$

■ Optimal C++ Code

```
int search(vector<int>& nums, int target) {
    int l = 0, r = nums.size() - 1;
    while (l <= r) {
        int mid = l + (r - l) / 2;
        if (nums[mid] == target) return mid;
        if (nums[l] <= nums[mid]) {
            if (nums[l] <= target && target < nums[mid])
                r = mid - 1;
            else
                l = mid + 1;
        } else {
            if (nums[mid] < target && target <= nums[r])
                l = mid + 1;
            else
                r = mid - 1;
        }
    }
    return -1;
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 5 Koko Eating Bananas

Pattern: Binary Search on Answer

LeetCode: #875

■ Input	piles and hours h
■ Output	minimum eating speed to finish in h hours
■ Constraints	positive piles
■ Main Idea / Pattern	Binary-search the speed. Check whether a candidate speed finishes within h.
■ Time Complexity	$O(n \log M)$
■ Space Complexity	$O(1)$

■ Optimal C++ Code

```
int minEatingSpeed(vector<int>& piles, int h) {
    int l = 1, r = *max_element(piles.begin(), piles.end());
    while (l < r) {
        int mid = l + (r - l) / 2;
        long long hours = 0;
        for (int p : piles)
            hours += (p + mid - 1) / mid;
        if (hours <= h) r = mid;
        else l = mid + 1;
    }
    return l;
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 6 Capacity To Ship Packages Within D Days

Pattern: Binary Search on Answer

LeetCode: #1011

■ Input	weights and days
■ Output	minimum ship capacity
■ Constraints	positive weights
■ Main Idea / Pattern	Search capacity from max weight to total weight; greedily count days for each capacity.
■ Time Complexity	$O(n \log S)$
■ Space Complexity	$O(1)$

■ Optimal C++ Code

```
int shipWithinDays(vector<int>& weights, int days) {
    int l = *max_element(weights.begin(), weights.end());
    int r = accumulate(weights.begin(), weights.end(), 0);

    while (l < r) {
        int cap = l + (r - l) / 2;
        int usedDays = 1, load = 0;

        for (int w : weights) {
            if (load + w > cap) {
                usedDays++;
                load = 0;
            }
            load += w;
        }

        if (usedDays <= days) r = cap;
        else l = cap + 1;
    }
    return l;
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 7 Search in Rotated Sorted Array

Pattern: Binary Search

LeetCode: 33

Input	nums, target
Output	index of target in rotated sorted array
Constraints	1 <= nums.size() <= 5000; values are distinct
Main Idea / Pattern	At least one half is sorted. Decide which half contains the target and discard the other.
Time Complexity	O(log n)
Space Complexity	O(1)

Optimal C++ Code

```
class Solution {
public:
    int search(vector<int>& nums, int target) {
        int left = 0;
        int right = nums.size() - 1;
        while (left <= right) {
            int mid = left + (right - left) / 2;
            if (nums[mid] == target) return mid;
            if (nums[left] <= nums[mid]) {
                if (nums[left] <= target && target < nums[mid]) {
                    right = mid - 1;
                } else {
                    left = mid + 1;
                }
            } else {
                if (nums[mid] < target && target <= nums[right]) {
                    left = mid + 1;
                } else {
                    right = mid - 1;
                }
            }
        }
        return -1;
    }
};
```

Practice: Re-solve without looking, then change one constraint and explain which part of the pattern changes.

PROBLEM 8 Find First and Last Position

Pattern: Boundary Binary Search

LeetCode: 34

Input	nums, target
Output	first and last index of target
Constraints	0 <= nums.size() <= 1e5
Main Idea / Pattern	Run binary search twice: once for the first position and once for the last.
Time Complexity	O(log n)
Space Complexity	O(1)

Optimal C++ Code

```
class Solution {
    int first(vector<int>& a, int target) {
        int l = 0, r = a.size() - 1, ans = -1;
        while (l <= r) {
            int m = l + (r - l) / 2;
            if (a[m] >= target) {
                if (a[m] == target) ans = m;
                r = m - 1;
            } else {
                l = m + 1;
            }
        }
        return ans;
    }
    int last(vector<int>& a, int target) {
        int l = 0, r = a.size() - 1, ans = -1;
        while (l <= r) {
            int m = l + (r - l) / 2;
            if (a[m] <= target) {
                if (a[m] == target) ans = m;
                l = m + 1;
            } else {
                r = m - 1;
            }
        }
        return ans;
    }
public:
    vector<int> searchRange(vector<int>& nums, int target) {
        return {first(nums, target), last(nums, target)};
    }
};
```

Practice: Re-solve without looking, then change one constraint and explain which part of the pattern changes.

PROBLEM 9 Find Minimum in Rotated Sorted Array

Pattern: Binary Search

LeetCode: 153

Input	nums
Output	minimum value in a rotated sorted array
Constraints	1 <= nums.size() <= 5000; values are distinct
Main Idea / Pattern	Compare mid with right to decide whether the minimum is to the left or right.
Time Complexity	O(log n)
Space Complexity	O(1)

Optimal C++ Code

```
class Solution {
public:
    int findMin(vector<int>& nums) {
        int left = 0;
        int right = nums.size() - 1;
        while (left < right) {
            int mid = left + (right - left) / 2;
            if (nums[mid] > nums[right]) {
                left = mid + 1;
            } else {
                right = mid;
            }
        }
        return nums[left];
    }
};
```

Practice: Re-solve without looking, then change one constraint and explain which part of the pattern changes.

PROBLEM 10 Koko Eating Bananas

Pattern: Binary Search on Answer

LeetCode: 875

Input	piles, h
Output	minimum eating speed to finish within h hours
Constraints	1 <= piles.size() <= 1e4; h >= piles.size()
Main Idea / Pattern	Speed is feasible monotonically. Binary search the smallest speed whose required hours <= h.
Time Complexity	O(n log maxPile)
Space Complexity	O(1)

Optimal C++ Code

```
class Solution {
public:
    int minEatingSpeed(vector<int>& piles, int h) {
        int left = 1;
        int right = *max_element(piles.begin(), piles.end());
        while (left < right) {
            int mid = left + (right - left) / 2;
            long long hours = 0;
            for (int p : piles) {
                hours += (p + mid - 1) / mid;
            }
            if (hours <= h) {
                right = mid;
            } else {
                left = mid + 1;
            }
        }
        return left;
    }
};
```

Practice: Re-solve without looking, then change one constraint and explain which part of the pattern changes.